
AI Engineer - Round 1 Assignment

AI-Assisted Prototype Challenge

AI Transformation & Innovation Team

IIFL Finance

Expected effort	2-3 hours maximum
Primary objective	Build a small working AI application and demonstrate sound engineering judgment.
AI tools	ChatGPT, Claude, Cursor, Copilot and similar AI coding agents are explicitly encouraged.
Submission	A GitHub repository with working code, sample data and a concise README.

What we want to learn

This exercise is intentionally time-boxed. We are not looking for production infrastructure or a polished product. We want to understand how you use an LLM, structure a simple AI workflow, validate AI-generated output, handle basic failure cases, and communicate engineering trade-offs clearly.

Important: Please do not spend more than three hours on this assignment. If you identify useful enhancements that do not fit within the time box, describe them briefly in the README instead of implementing them.

1. Assignment

Build a Policy-Aware Customer Support Agent

You will be provided with:

- 3 short policy / FAQ documents; and
- 5 sample customer questions.

Build a small application that accepts a customer question and produces an answer grounded in the provided policy information.

Required workflow

Your application should:

1. Accept a customer question.
2. Identify or retrieve the most relevant policy / FAQ content.
3. Generate an answer grounded in that content.
4. Return a structured result.
5. Escalate the query when the available information is insufficient or confidence is low.

Expected structured output

```
{
  "query": "...",
  "category": "...",
  "answer": "...",
  "source": "...",
  "confidence": "high | medium | low",
  "action": "respond | escalate"
}
```

2. Minimum Technical Requirements

Requirement	What is expected
LLM usage	Use any suitable LLM or AI API, such as OpenAI, Claude, Gemini, Llama or another model of your choice.
Retrieval / tool step	Implement at least one function, tool or lookup that selects relevant information from the provided policy documents. A vector database is not required.
Structured output	Return JSON or another clearly structured result that can be inspected programmatically.
Error handling	Handle at least one failure case, such as empty input, missing content or a query that cannot be answered confidently.

Keep the implementation deliberately small

A Python script, CLI, small API or basic Streamlit app is sufficient. Plain Python is completely acceptable. We value a working, understandable solution over framework complexity.

3. Out of Scope

You do **not** need to build authentication, a database, cloud deployment, Docker / Kubernetes infrastructure, production monitoring, a complex multi-agent architecture, a vector database, or a polished UI.

4. Deliverables

Submit a GitHub repository containing the following at minimum:

```
README.md
src/
data/
```

1. Working code

The application should run locally with clear setup instructions and should work on the supplied examples.

2. Concise README

Keep the README short and answer the following:

1. **How does your solution work?** - Maximum 5-8 lines.
2. **Why did you choose this model / framework / approach?** - Maximum 3-5 lines.
3. **What would you improve before production use?** - Maximum 5 bullets.
4. **What is one important security or governance concern in financial services?** - Maximum 3-5 lines.
5. **What AI coding tools did you use, and how did you use them?** - Maximum 3-5 lines.

Include **one example input and output**. A separate report, architecture document and video recording are not required.

5. Evaluation

Area	Weight	What we look for
Working solution	40%	Core workflow works correctly for the provided examples.
AI engineering approach	25%	Sensible LLM usage, retrieval / tooling, prompting and structured output.
Code quality	15%	Implementation is readable, reasonably organized and easy to run.
Engineering judgment	10%	Candidate recognizes limitations and sensible production next steps.
Security & enterprise awareness	10%	Candidate identifies a meaningful security, privacy or governance concern.

6. Suggested Time Box

Activity	Guide	Focus
Understand & set up	15-20 min	Review inputs, choose approach, create project skeleton.
Implement	60-90 min	Build retrieval, LLM call, structured response and escalation logic.
Test & fix	20-30 min	Validate supplied examples and at least one failure case.
README & cleanup	20-30 min	Add run instructions, one example and short engineering notes.

Round 2 discussion: Shortlisted candidates may be asked to explain the design, discuss productionization and security trade-offs, or extend the prototype live. The Round 1 submission is therefore expected to be a focused prototype, not a production-ready system.

All the best. We look forward to seeing how you think, build and use AI tools effectively.